

Informe

Proyecto DOO

Numero de Grupo: 14

Nombres: Matias Ignacio Cabello Carvajal

Maximiliano Enrique Pinto Valenzuela

Paulo Bastian Torres Thieme

Fecha: 8-7-2026

Profesor: Geoffrey Hecht

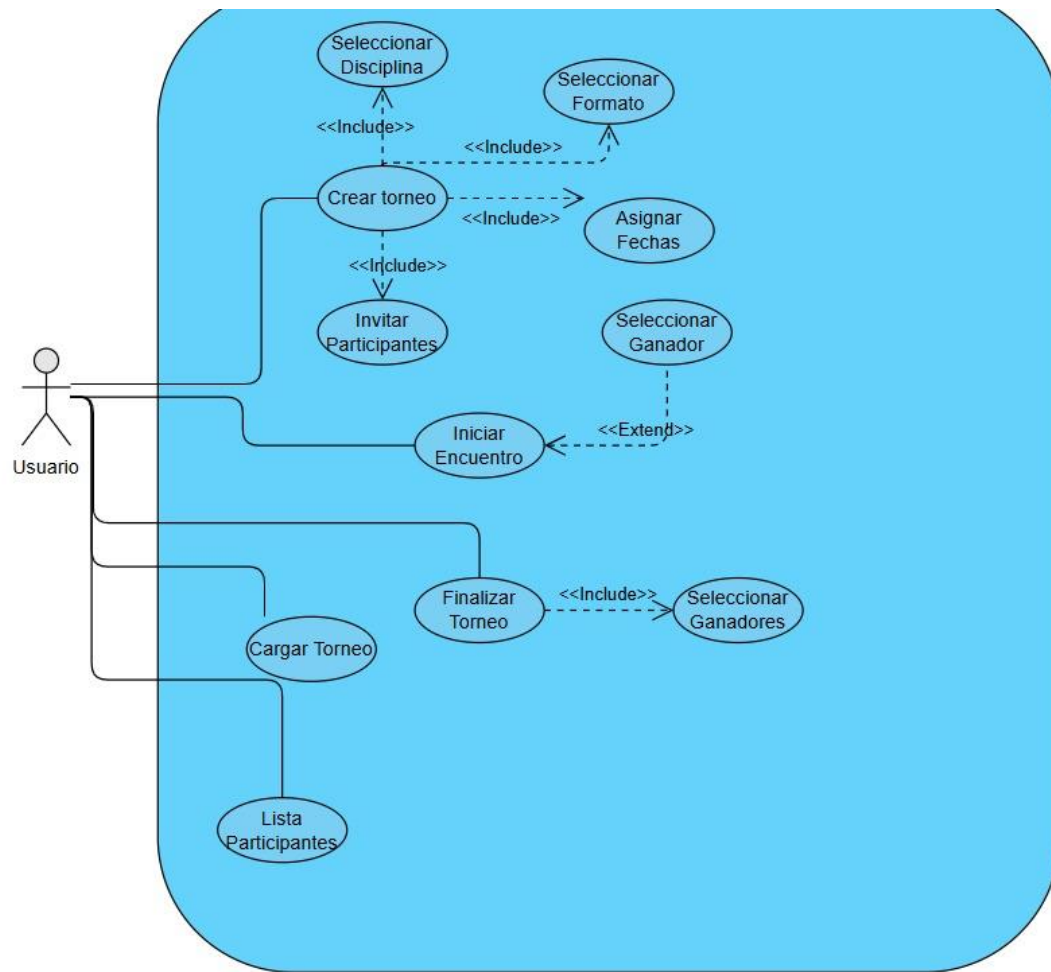
Referente: Bryan Fuentes

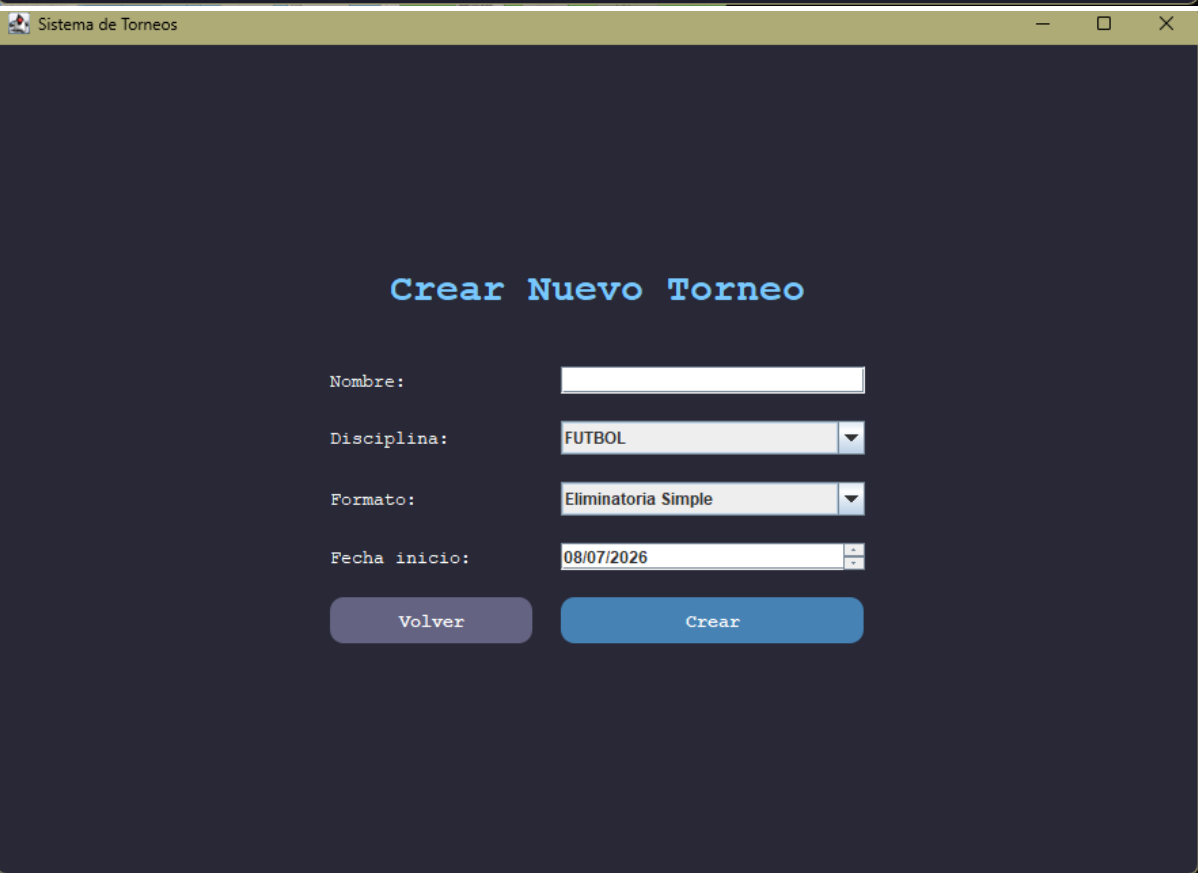
Enunciado de Proyecto:

Tema 1: Sistema de Gestión de Torneos (Referente: Bryan Aguirre)

“Este sistema está diseñado para facilitar la organización de torneos deportivos o de juegos. Permitirá a un organizador definir las características del torneo, como el nombre, la disciplina (ej. fútbol, ajedrez, videojuegos), las fechas y un formato principal (como eliminatoria directa, eliminatorio doble, liga simple...). Se podrán inscribir participantes, ya sean jugadores individuales o equipos, almacenando información básica como nombres y datos de contacto. El sistema deberá ser capaz de generar un calendario de enfrentamientos o un bracket inicial basado en los inscritos y el formato. Durante el torneo, se registrarán los resultados de cada enfrentamiento, lo que actualizará automáticamente las posiciones, el avance en el bracket o las tablas de clasificación. Los usuarios podrán visualizar el estado actual del torneo, los próximos encuentros y las estadísticas generales.”

Diagrama de Casos de Uso:





hubiera encapsulación en lo que es como se generan y avanzan las rondas dentro de del programa.

Tratamos de guiarnos por el principio abierto/cerrado para que siempre que se quiera añadir más formatos de torneo al proyecto, solo se tenga que añadir una nueva clase y cumplir una lista de reglas base, sin modificar el código de Torneo.

Usamos el patrón para ocultar como se emparejan y se maneja matemáticamente los emparejamientos dentro del formato. La clase de Torneo solo se encarga de delegar las responsabilidades de generar las llaves, ver los siguientes encuentros y escoger al ganador.

Clases:

- Formato (Interfaz): Define las reglas y operaciones que deberán seguir todos los formatos de Torneo que se usen en el Proyecto.
- ElimDirecta, ElimDoble, LigaSimple (Formatos Especificos): Implementan la interfaz reescribiendo la lógica de Formato, las fórmulas matemáticas y las reglas específicas de cada estilo de torneo.
- Torneo: Mantiene una referencia a la interfaz Formato. Delega el ejecutar de las reglas a la interfaz, sin saber por completo los detalles internos de si se está jugando una liga o una llave de eliminación.

Patrón 2: Observer (Observador / Eventos)

Hacía falta un mecanismo para que automáticamente se hicieran los cambios de estado de las partidas en el Torneo, pero sin acoplar demasiado. Decidimos usar este patrón para evitar que la clase principal “chequeara” cíclicamente el estado de cada partida, lo que no era ideal y complicaba el código.

Lo usamos para supervisar el flujo de las rondas. Cuando una Partida registra un resultado, se utiliza un gestor interno para mandar una alerta automática. El Torneo suscrito a los eventos, recibe la notificación al instante, procesa el resultado en su código y verifica si todas las partidas de la ronda actual terminaron. Si es así, el torneo llama a la lógica para generar la siguiente fase o en el último caso, escoger al Campeón.

Clases relacionadas:

ObservadorPartida (Interfaz Observador): Define el contrato con el método actualizar (Partida partida) que se debe implementar por quienes lo necesiten usar.

GestorEventosPartida (Sujeto): Maneja la lista de suscripciones, agregando, eliminando y notificando a los observers sin alterar la lógica de la partida.

Partida (Sujeto Concreto): Cambia su estado (ej. de PENDIENTE a TERMINADA) y delega en el gestor la tarea de avisar a los suscritos.

Torneo (Observador Concreto): Implementa la interfaz para escuchar a las partidas y ejecutar el método `continuarTorneo()` cuando sea necesario.

Patrón 3: Composite (Compuesto)

El sistema tenía que soportar competiciones de distintas Disciplina (Ajedrez, Futbol, Bàsquet y así). Elegimos este patrón estructural para evitar que se duplicara el código y no tener que usar validaciones constantes (`if(esEquipo)`) a la hora de organizar las rondas. Así logramos tratar entidades individuales y agrupaciones bajo una misma lógica.

El patrón se usa para la inscripción y gestión de competidores. Gracias a este patrón, el Torneo (y los algoritmos de las llaves) simplemente operan sobre la abstracción de Participante. El sistema empareja, avanza y define ganadores sin que importe si el objeto subyacente es una sola persona o un grupo de personas. Además, permite que un Equipo sobrescriba validaciones globales basándose en sus componentes internos (por ejemplo, evaluar que el equipo tenga integrantes antes de declarar que "está listo para jugar").

Clases relacionadas:

Participante (Componente): Clase abstracta que define el comportamiento base e interfaz común para todos los competidores (nombres, contactos, validaciones de estado).

Jugador (Hoja): Entidad atómica y singular que contiene datos específicos (como el RUT) y que no posee subelementos.

Equipo (Compuesto): Extiende desde el componente principal, pero internamente administra una lista (`List<Jugador>`), aplicando restricciones de límite máximo de integrantes según lo que pide la disciplina a jugar en el Torneo.

Decisiones y Cambios en el camino:

Partimos queriendo tener Brackets gráficos como los que se ocupan en torneos de E-sports, pero al darnos cuenta de que habría que considerar diferentes tamaños de grupos (no los normales de 8 o 16), se volvía complicado el ubicar correctamente los elementos gráficos en el apartado visual.

Al final acabamos decidiendo usar un sistema de calendario de partidas, en el que pudieramos ver las partidas que se jugaban dentro de una misma ronda y estuviera

bien organizado y reactivo. De poder tener más tiempo nos hubiera gustado intentar hacerlo, pero con la experiencia que tenemos actualmente con Swing, no logramos hacerlo funcionar para casos generales o no ideales.

Otra idea que se nos ocurrió fue tener un sistema que guardara los cambios dentro del Torneo en un archivo y se pudiera cargar una partida o continuarla de esa forma.

Debido a que esto nos pedía aprender a trabajar con ingreso y salida de archivos de texto en el código y que se pudiera leer, optamos por otro método más simple, pero básico de conservar cambios. En el programa, se puede crear un Torneo, así como también ver uno activo (que se debe empezar previamente mediante la opción de crear torneo).

Problemas y Autocritica:

El principal problema que enfrentamos fue de falta de experiencia usando el apartado gráfico, funciones externas a las ya usadas dentro de la asignatura, así como también problemas de tiempo y coordinación por la agenda ocupada de final de semestre, en la que había varias semanas acopladas con certámenes y trabajos que complicaba reunirse con tiempo adecuado y causó que hubiera un trabajo más asincrónico que el que queríamos tener.

Dentro de este trabajo, en el repositorio tuvimos errores comunes de merge por el mismo trabajo que era asincrónico y no permitía coordinarnos bien en el caso de tratar de hacer commits en corto plazo, cosas que si solucionamos en la etapa final del proyecto.